3. We run the Q-network forward.
4. We're using an epsilon-greedy implementation, so at time *t* with probability ε we will choose a random action. With probability 1 – ε, we will choose the action associated with the highest Q value from our neural network.
5. Take action *a* as determined in the preceding step, and observe the new state s′ and reward r_{t+1}.
6. Run the network forward using s′. Store the highest Q value, which we'll call max Q.
7. Our target value for training the network is $r_{t+1} + \gamma * \max Q_A(S_{t+1})$, where γ (gamma) is a parameter between 0 and 1. If after taking action a_t the game is over, there is no legitimate s_{t+1}, so $\gamma * \max Q_A(S_{t+1})$ is not valid and we can set it to 0. The target becomes just r_{t+1}.
8. Given that we have four outputs and we only want to update (i.e., train) the output associated with the action we just took, our target output vector is the same as the output vector from the first run, except we change the one output associated with our action to the result we computed using the Q-learning formula.
9. Train the model on this one sample. Then repeat steps 2–9.

To be clear, when we first run our neural network and get an output of action values like this,

```
array([[-0.02812552, -0.04649779, -0.08819015, -0.00723661]])
```

our target vector for one iteration may look like this:

```
array([[-0.02812552, -0.04649779, 1, -0.00723661]])
```

Here we just changed a single entry to the value we wanted to update.

There's one other detail we need to include in the code before we move on. The Gridworld game engine's `makeMove` method expects a character such as *u* to make a move, but our Q-learning algorithm only knows how to generate numbers, so we need a simple map from numeric keys to action characters: